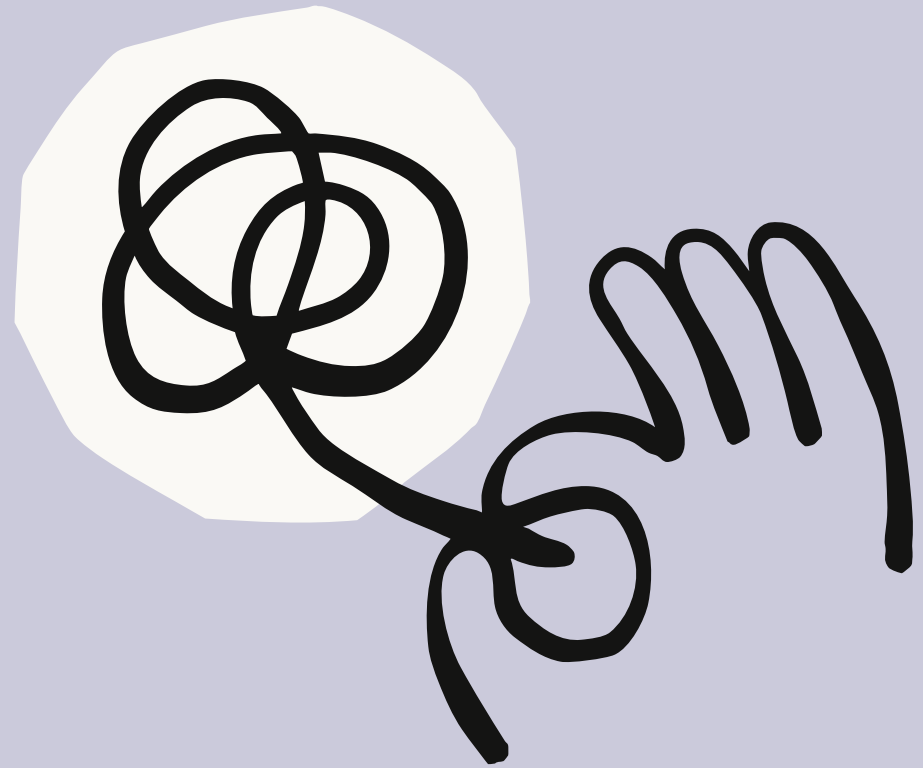


The Founder's Playbook: Building an AI-Native Startup



Chapter 4

MVP Stage

MVP Stage

Plenty of founders treat the MVP stage as a construction phase, but the MVP stage is still fundamentally an evidence-gathering exercise. The difference is that you are now gathering evidence about the solution instead of the problem space; specifically, whether a real, identifiable group of people finds it valuable enough to use it, return to it, pay for it, and/or tell others about it.

MVP stage goals

As the founder of an AI-native startup, your goal is to **translate a validated problem into a working product** that real users will actually use. This is not the full version with every roadmap feature but the smallest, most focused iteration of your idea that puts a real solution in front of real users and generates real evidence of product-market fit.

At the same time, how you build now determines what's possible later. This means that the MVP stage has a second, equally important goal of **moving fast without accruing the type of technical debt** that compounds—and will haunt you the moment real users arrive in meaningful numbers.

And finally, **investing in persistent context** from day one is what keeps AI a force multiplier instead of a source of entropy. In an AI-native startup, your codebase is something you collaborate with AI on session after session, which makes legibility foundational. Founders who skip specs, architectural decisions, and context files (like CLAUDE.md) hit a predictable wall where every new session requires re-explaining the codebase and AI-generated changes drift from the original vision.

MVP stage exit criteria

The MVP stage exit condition is genuine evidence of product-market fit: proof that a specific, identifiable group of users has found the product valuable enough to return to it (retention), pay for it (revenue), or tell others about it (referral).

MVP stage challenges

In the MVP stage, a founder's prime directives are speed and judgment. The challenges here center on whether you can build the right thing, the right way, fast enough to matter, without cutting corners that will cost you later.

Agentic technical debt

***The challenge:** Because AI essentially removes every natural bottleneck that once controlled what reaches production, speed is guaranteed. But when speed is the only variable that founders factor into their MVP build, they risk accruing technical debt they'll struggle to pay off.*

Some technical debt is appropriate at the MVP stage, with the understanding that it must be managed before scaling. It builds gradually and can be cleared over time or in a dedicated sprint. AI technical debt, however, compounds.

Without specs and architectural constraints written down somewhere the AI can read, each session re-derives foundational decisions from scratch, and those decisions drift. You end up with a codebase that has no coherent mental model behind it, not because any single piece is bad, but because the pieces were never designed to fit together. That's a real problem, and it does tend to surface late.

Falling for false product-market fit

The challenge: *AI tools can generate impressive early numbers, but these are not a guarantee that the market needs your product.*

Early momentum is one of the most psychologically powerful experiences a founder can have. After weeks or months of validation work and careful, disciplined building, shipping a product feels like confirmation that you were right all along.

Agentic coding tools can help you reach this moment faster than ever before, but early traction is not the same as product-market fit. Launch energy is generated from ephemeral forces, like your founder's friends, prospective buyers at your investor's other portfolio companies, or a Hacker News headline that drives a spike. Unfortunately, none of these reliably predicts what happens at week six or week twelve when that initial boost has faded.

Zero-friction scope creep

The challenge: *When building feels effortless and is nearly free, there's always one more cool feature to add or one more edge case to handle. This scope creep can do more harm than good.*

Scope creep has always been a startup risk. The difference now is that the traditional forcing function against it—the real cost of engineering time—no longer exists in the same way when adding a feature takes an afternoon instead of a sprint.

The challenge here is that each individual addition is defensible. Of course the product should handle that edge case; of course users will want that workflow. These don't feel like scope creep in the moment because each one takes so little effort to build with agentic coding, but as your product sprawls beyond its original boundaries you risk losing direction and momentum.

The antidote is a written scope definition created before building begins, describing what the product does, what it deliberately does not do, and the specific evidence from real users that would justify adding something new. This moves the decision point from "should we build this?" to "a critical mass of users have told us they can't get value from the product without this?"

Insecure by inexperience

The challenge: *Founders using AI tools to rush applications to market without first understanding fundamental security principles end up exposing their users to preventable risks.*

The hard truth is that agentic coding tools generate code that works, not code that is inherently secure. Functional code is easy, because either the feature works or it doesn't. Security vulnerabilities are invisible until they're exploited, which means there's no natural feedback loop to alert a first-time founder that something is wrong. Shipping a live MVP to real users, however, means real data, real exposure, and real consequences if something goes wrong.

Slighting security isn't brand new to AI-native projects. Bootstrapped startups in every era often delayed security considerations until late in the build, sometimes waiting until the verge of production launch. A security review before any user touches your app or solution is the minimum responsible threshold for releasing a minimum viable product into the world.

How Claude can help MVP stage founders

Define your architecture before you build

Before Claude Code writes a line of production code, use Claude to define and document the architectural decisions that will govern everything built in this stage: the patterns to follow, the dependencies to avoid, the tradeoffs being made and why. This output will serve as a focused architectural context document and establish the guardrails that Claude Code will operate inside.

Without this context, each session starts from scratch and Claude Code is forced to infer its own structural assumptions. Letting Claude Code build without guardrails produces a codebase that will be functional but structurally incoherent, and iterating on and scaling incoherent codebases is ultimately a waste of time and tokens. Sooner or later there's a point where the code inevitably collapses, forcing you to rebuild from scratch.

- **Exercise:** Before opening Claude Code, open Claude and describe what you're building: the core problem it solves, the users it serves, and the scale you realistically expect in the next six months. Ask it to help you define the architectural principles that should govern your MVP build, the dependencies to avoid given your constraints, and the tradeoffs you're consciously accepting at this stage.

Next, save this output as CLAUDE.md markdown file(s). This is your architectural context document: the first artifact of your build, and the one every subsequent session depends on. CLAUDE.md files serve as project-level instructions for Claude Code, providing project-specific context and instructions that are automatically read by the Agent SDK when it runs in a directory. Functionally, they are persistent "memory" for your project.

Define and enforce your MVP scope

Scope creep without friction is one of the defining failure modes of AI-era MVPs. Just as you defined and documented your product's application architecture, you also need to define your MVP's scope before a single feature gets built.

Claude can help you create a scope document that describes what your MVP product does, what it deliberately does not do, and feature amendment criteria: what specific evidence from real users would justify adding something new at this point.

When new feature ideas surface—and they surely will—you use Claude to pressure-test whether it's genuine signal from users or founder enthusiasm dressed up as product thinking.

Build your MVP with Claude Code

Once architecture and scope are defined, Claude Code becomes the primary MVP build tool. Use it to generate, test, debug, and iterate on your codebase, but treat each session as an execution of product decisions you have already made, not as an opportunity to throw in some new ones.

Start each Claude Code session by (1) revisiting your scope document and (2) providing the model with your CLAUDE.md architectural context document. End each session by updating it with any decisions the session surfaced. The goal is a codebase whose structure you can explain, not just a codebase that runs.

- **Exercise:** Create a simple session template for your Claude Code work that includes the architectural context document, the specific task for this session, and any constraints or patterns to observe. At the end of each session, add a brief log entry to the context document that details what was built, what decisions were made, and what assumptions the session introduced. Five minutes of documentation per session is cheap insurance against architectural drift that compounds into an unmanageable codebase.

Security review before any user touches it

As an AI-native startup founder, your responsibility is to know what's in your codebase, understand any potential exposure vectors, and not ship obvious vulnerabilities to real users who are trusting you with their data.

Claude can do a useful first-pass security review of AI-generated code and can help identify common vulnerabilities. It's a good habit to build into the loop before shipping. It is not a substitute for security tooling, however, or, at higher stakes, a human reviewer—and founders who treat it as one are the ones who end up in the breach stories.

Claude Code Security goes further: it scans codebases for security vulnerabilities and suggests targeted patches for human review, surfacing issues that traditional methods can miss.

Note: At the time of this ebook's publication, Claude Code Security is a limited beta release so check current availability before building it into your workflow.

- **Exercise:** Before deploying to any real users, run your core application code through Claude with a specific brief: review for authentication and session handling, data exposure in API responses, input validation and injection risks, and dependencies with known vulnerabilities. Treat each finding seriously and assess whether it requires a fix, with human review for anything that touches authentication, secrets, or data handling.

Build your measurement framework *before* launch

The founders who mis-identify early traction as product-market fit are typically the same ones who started tracking data after launch, using metrics chosen to assess what was working rather than to surface what wasn't. The antidote is to establish your measurement framework before the first user shows up.

Use Claude to define which metrics matter for your specific product, what the benchmarks are, and what patterns in the data would constitute genuine product-market fit versus flattering noise. Specifically: set your retention benchmarks, your activation criteria, and your Day 7 and Day 30 targets before releasing your MVP.

Next, define what a false positive looks like for your specific product: signups without activation, revenue without retention, or initial enthusiasm without repeat usage, for example. When the data arrives, ask Claude to make the adversarial case against your own traction: what would a skeptic say about these numbers?

Manage discovery and user feedback logistics

Once real users are in the product, the operational layer expands fast. Claude Cowork handles the important-but-tedious work like building and maintaining user contact lists, running outreach sequences, scheduling feedback sessions, triaging bug reports, and tracking iteration cycles. The same MCP integrations that managed discovery logistics in the Idea stage apply here.

Keep a human in the collection loop for nuanced exploration of user feedback. A user saying, for example, "this is great but I wish it could also...", requires interpretation: Is it a core need or a nice-to-have? Is it specific to this customer or representative of a segment? Is the missing feature the real problem, or is something upstream in the onboarding? No tool can answer those questions.

- **Exercise:** Configure Claude Cowork to run your MVP-stage feedback loop: draft outreach to your early user list, schedule feedback sessions, design structured intake process for bug reports and feature requests, and write up a weekly synthesis of what's come in. Review the synthesis yourself first; after that, you can ask Claude to analyze the information to catch any significant points you may have overlooked.

Iterate toward evidence, not toward completeness

The MVP stage ends when you have genuine evidence of product-market fit, no matter how "finished" the product feels. Declaring that you've achieved product-market fit and are now ready to move on from the MVP phase to the Launch stage is ultimately a judgement exercise that combines founder intuition with collected evidence. There are some useful litmus tests, though:

- **The Sean Ellis test:** Ask your active users: "How would you feel if you could no longer use this product?" If more than 40% answer "very disappointed," that's a meaningful PMF indicator.
- **The effort test:** Pre-product-market fit, retention requires constant intervention, including frequent outreach, incentives, personal follow-up, and a heroic founder energy expended to keep users engaged. Post product-market fit, the product starts doing that work on its own. When things begin pulling instead of pushing, that shift in effort is one of the clearest signals that something real has changed.

Ultimately, no single data point confirms product-market fit because it's a pattern that has to hold across multiple iteration cycles before you can definitively call it.

Pivot when the evidence demands it

What if, even after investing all this work, you just can't seem to arrive at product-market fit? The fact that your results don't confirm the direction you started from is not failure, it's the system working: the MVP stage is designed to surface this information before you over-invest in the wrong answer.

When the data doesn't support your current product, use Claude to work through what that data is telling you.

- **Exploring alternative customer segments.** Perhaps the users who aren't converting were never the right target to begin with. Often the right audience is already in your data, just underweighted.
- **Adjusting your product's value prop.** Maybe you have the correct audience but your MVP is just not resonating with users. An adjustment to onboarding, messaging, or core feature emphasis can potentially fix this without changing what you've built.

Stay open to the possibility that the disconnect may run deep enough to require a more fundamental change

- **Exercise:** If you've completed three or more iteration cycles without meaningful movement toward your product-market fit benchmarks, use Claude to run a diagnostic before deciding what to do next. Feed it your retention data, your user feedback, and your original problem hypothesis, and ask it three questions:
 - Is there a segment in this data responding differently than the rest?
 - Is the gap between designed value and experienced value a positioning problem or a product problem?
 - What would have to be true for the current product to find genuine PMF, and is that scenario realistic given what you're seeing?

Let the answers determine whether you adjust, pivot, or return to the Idea stage.